isolutions

Generative AI Solution Development with DBX

isolutions Tech Meetup
April 2025

isolutions

Introduction to RAG

Retrieval **A**ugmented **G**eneration, or RAG;

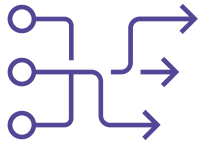
- Is a pattern (Lewis et al. 2020) that can improve the **efficacy** of large language model (LLM) applications by **leveraging custom data**.

- Is done by **retrieving** data/documents relevant to a question or task and providing them as context to **augment** the prompts to an LLM to improve **generation**.



isolutions

How do Language Models Learn Knowledge?



Model Pre-Training

- Training an LLM from scratch
- Requires large datasets (billions to trillions of tokens)



Model Fine Tuning

- Adapting a pre-trained LLM to specific data sets or domains
- Requires thousands of domain-specific or instruction examples



Passing Contextual Information

- Combining an LLM with external knowledge retrieval
- Requires external knowledge base
- How do we use vectors to **search** and provide **relevant context** to LLMs?

LESS Complexity and Compute-intensiveness

isolutions

Passing Context to LMs Helps Factual Recall

- **Passing context** as model inputs improves factual recall
  - Analogy: Take an exam with **open notes**

- LLMs are evolving to accept a larger/infinite input token window size



isolutions

RAG Use Cases

Q&A Chatbots

- Incorporate LLMs with chatbots to automatically derive more accurate answers.
- Used to automate customer support and website lead follow-up to answer questions and resolve issues quickly.

Search Augmentation

- Incorporating LLMs with search engines that augment search results with LLM-generated answers.
- Makes it easier for users to find the information they need.

Content Creation and Summarization

- Facilitate the development of high-quality articles, reports, and summaries using additional context.
- Example; generation of news articles or summarization of lengthy reports.

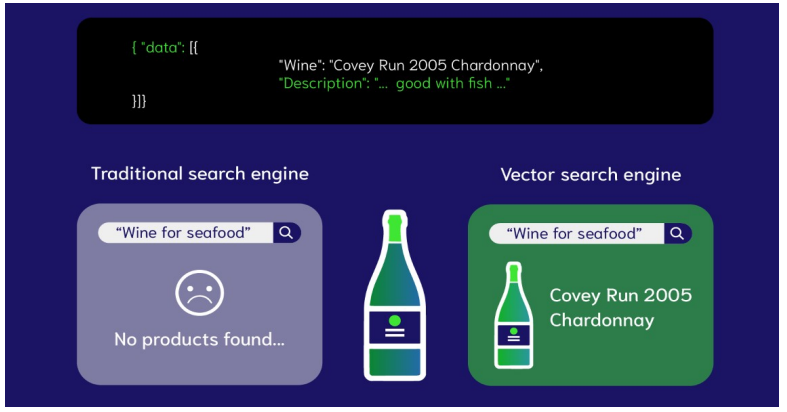
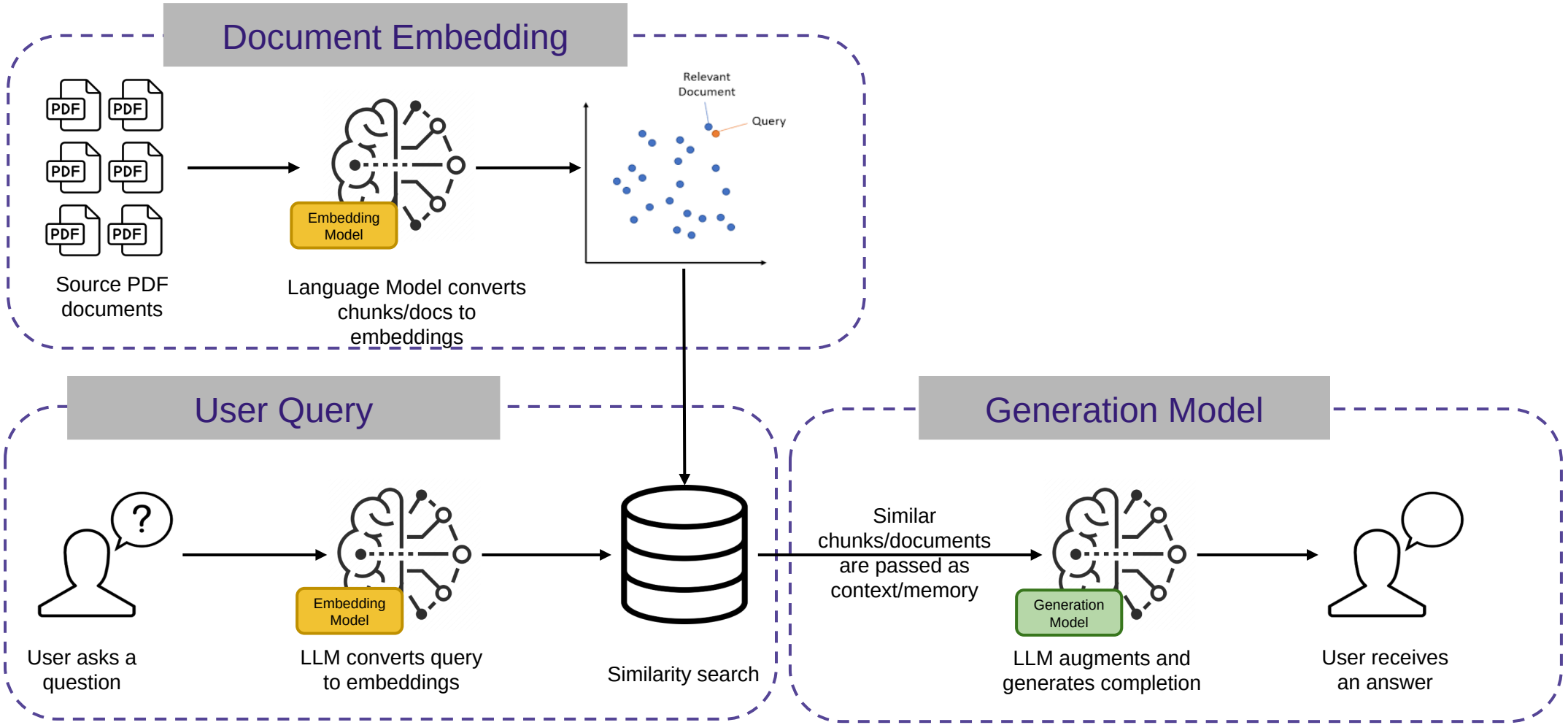


Image Source: Weaviate

isolutions

RAG Architecture



isolutions

Preparing Data for RAG Solutions

isolutions

Why is Data Prep Important for RAG?

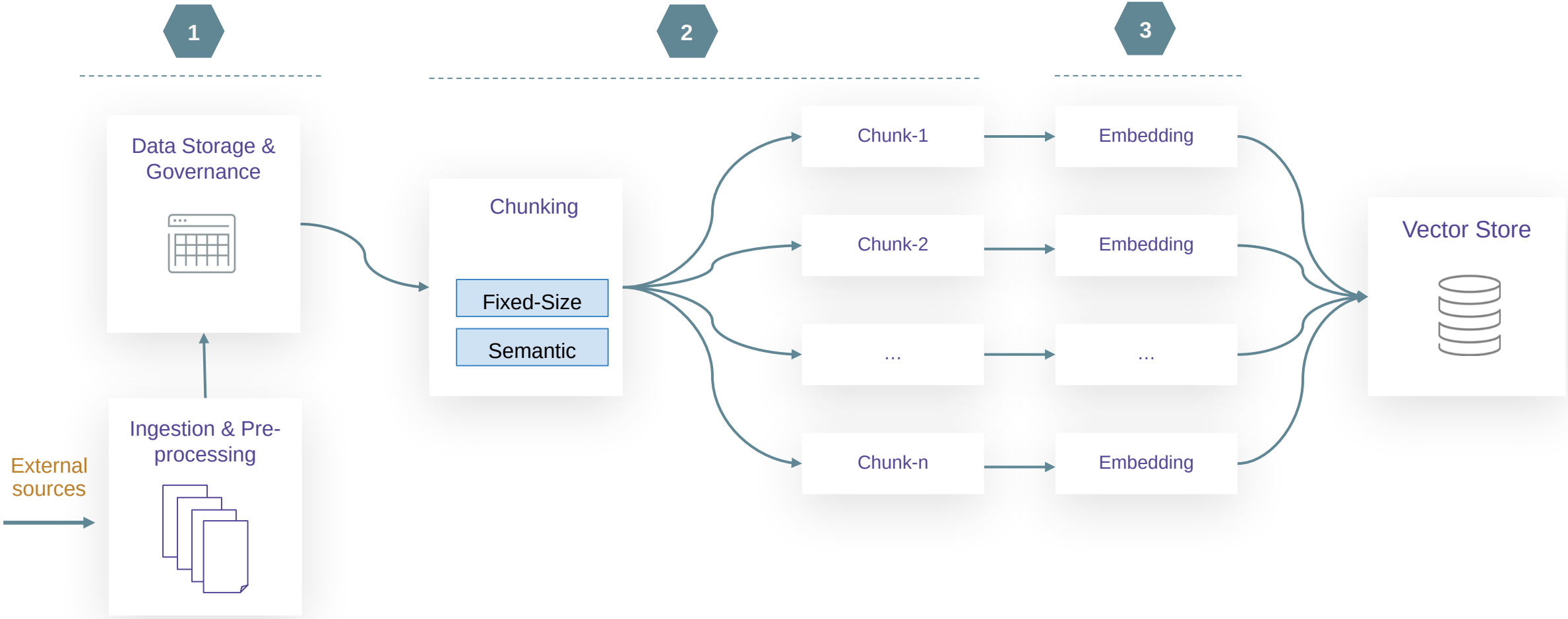
Potential **issues** when data is prepared improperly

- **Poor quality model output:** If data is inaccurate, incomplete, or biased, the RAG system is more likely to produce misleading or incorrect responses.
- **“Lost in the middle”**: In long context, LLMs tend to overlook the documents placed in the middle (Related Research Paper and needle in haystack test repo).
- **Inefficient retrieval**: Poorly prepared data would decrease the accuracy and precision of retrieving relevant information from knowledge base.
- **Exposing data**: Poor data governance could lead to exposing data during the retrieval process.
- **Wrong embedding model**: Wrong embedding model would decrease the quality of embeddings and retrieval accuracy.

isolutions

Data Prep Process Overview

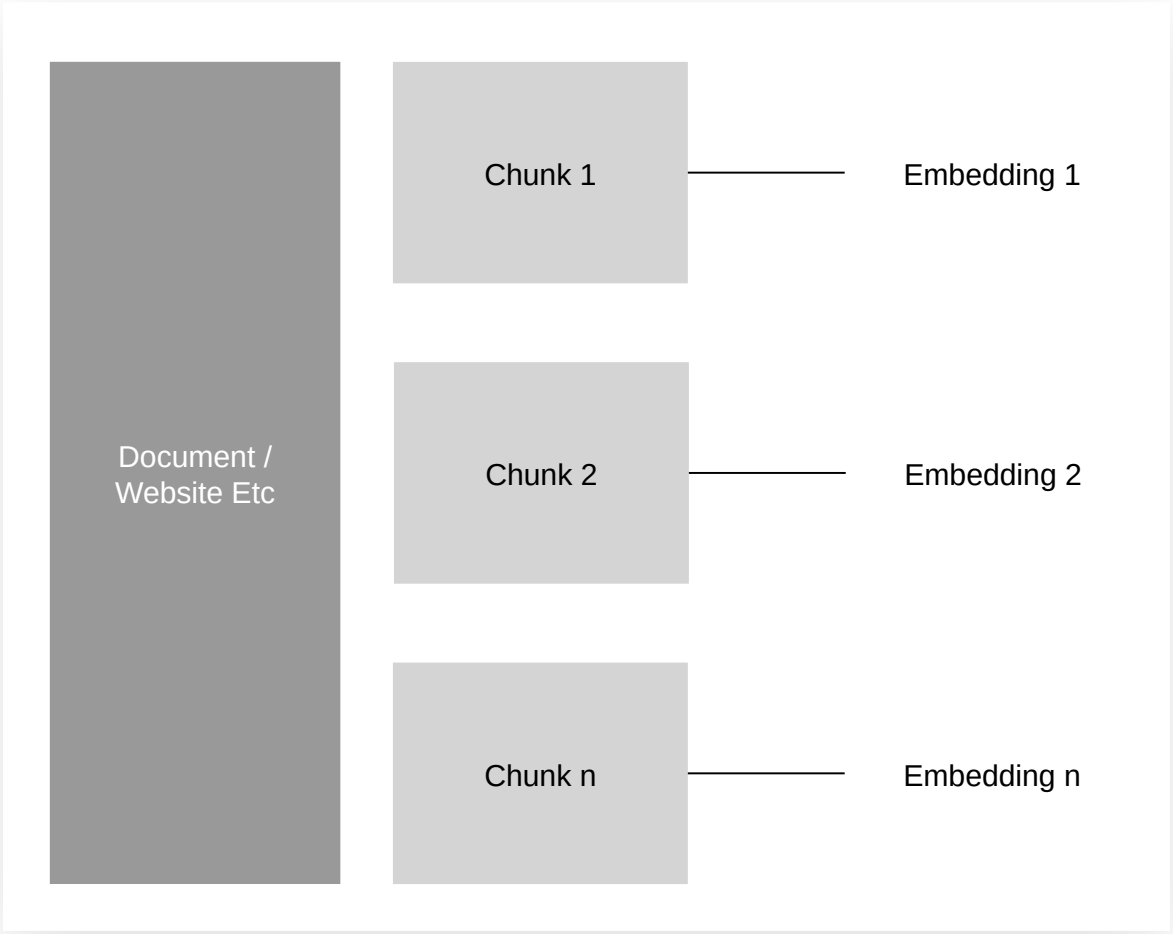
A simple data prep process



isolutions

Data/Document Extraction

Ingesting (text) documents and making them searchable



Typical Process:

1. Split documents into chunks
2. Embed the chunks with a model
3. Store them in a vector store

Constraints/Risks:

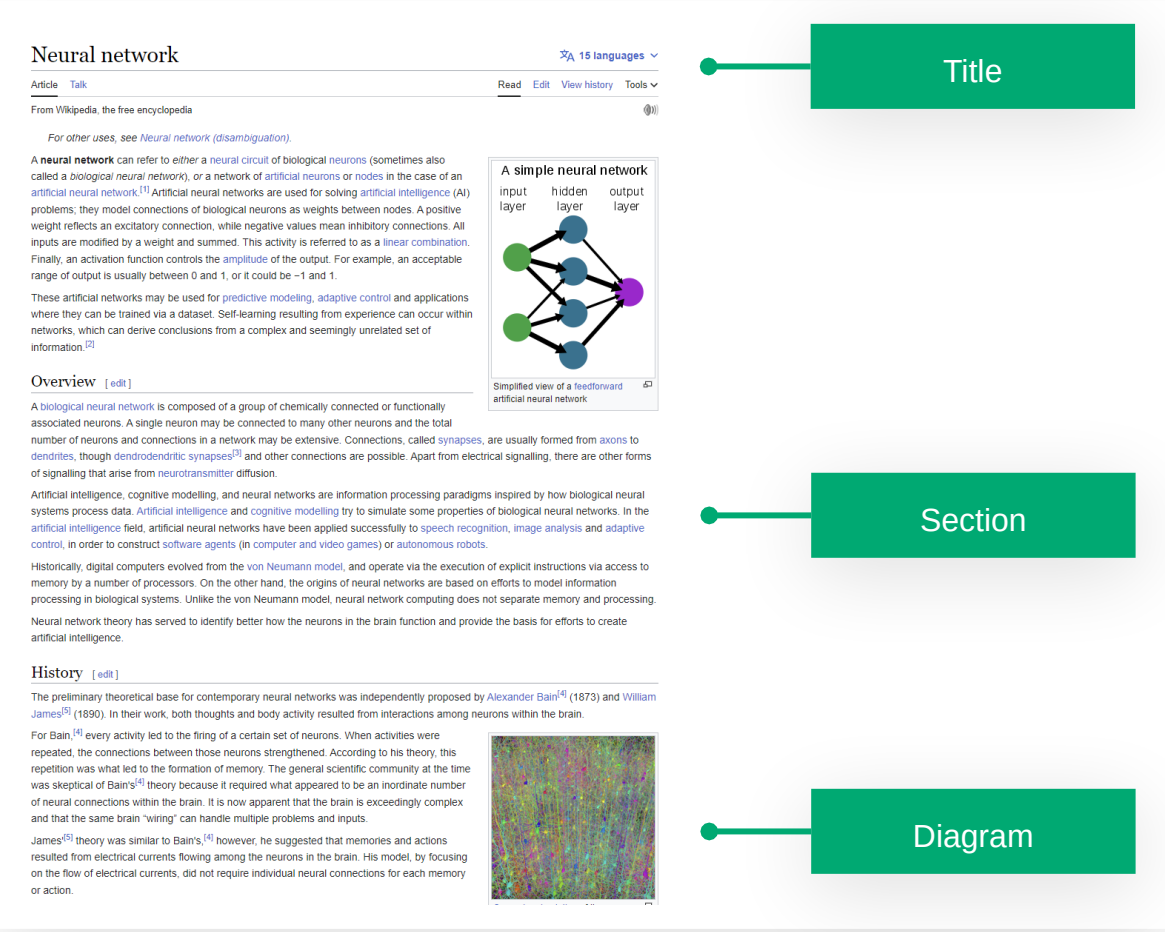
- **Chunks could be out of Context**
- **No overall logic/rigor**



isolutions

How to Chunk Data?

How should we organise it?



Semantic Chunking:

- Chunk by sentence/paragraph/section
- Leverage special punctuation (i.e. '.', '\n')
- Include/Inject metadata/tags/title(s)

&/OR

Fixed-size Chunking:

- Divide by a specific number of tokens
- Simple and computationally cheap method

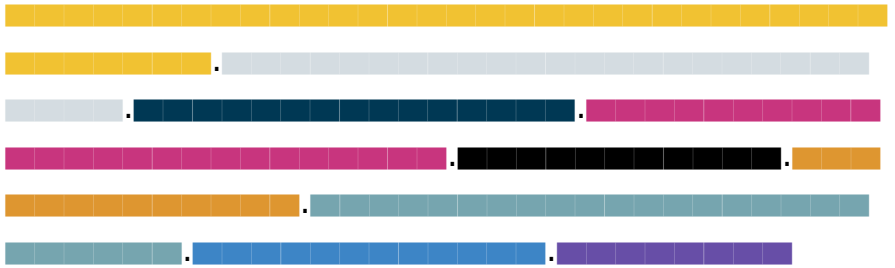
isolutions

Chunking Strategy

Another iterative step! Experiment with different chunk sizes and approaches

- How long are our documents?
  - 1 sentence?
  - N sentences?
- If 1 chunk = **1 sentence**, embeddings focus on specific meaning
- If 1 chunk = multiple paragraphs, embeddings capture broader theme
  - How about splitting by headers?

Chunking by sentence:



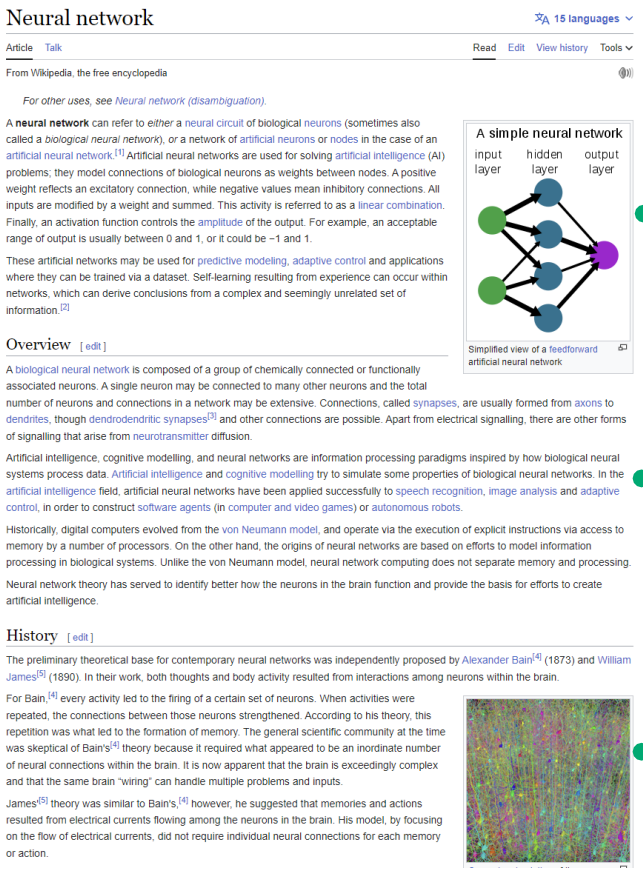
Chunking by Paragraph:

Use ChunkViz helper tool

isolutions

Advanced Chunking Strategies

Summarization



Neural network

15 languages

Article Talk Read Edit View history Tools

From Wikipedia, the free encyclopedia

For other uses, see Neural network (disambiguation).

A **neural network** can refer to *either* a neural circuit of biological neurons (sometimes also called a *biological neural network*), *or* a network of artificial neurons or nodes in the case of an artificial neural network.[1] Artificial neural networks are used for solving artificial intelligence (AI) problems; they model connections of biological neurons as weights between nodes. A positive weight reflects an excitatory connection, while negative values mean inhibitory connections. All inputs are modified by a weight and summed. This activity is referred to as a linear combination. Finally, an activation function controls the amplitude of the output. For example, an acceptable range of output is usually between 0 and 1, or it could be −1 and 1.

These artificial networks may be used for predictive modeling, adaptive control and applications where they can be trained via a dataset. Self-learning resulting from experience can occur within networks, which can derive conclusions from a complex and seemingly unrelated set of information.[2]

A simple neural network

input layer hidden layer output layer

Simplified view of a feedforward artificial neural network

Overview [edit]

A biological neural network is composed of a group of chemically connected or functionally associated neurons. A single neuron may be connected to many other neurons and the total number of neurons and connections in a network may be extensive. Connections, called synapses, are usually formed from axons to dendrites, though dendrodendritic synapses[3] and other connections are possible. Apart from electrical signalling, there are other forms of signalling that arise from neurotransmitter diffusion.

Artificial intelligence, cognitive modelling, and neural networks are information processing paradigms inspired by how biological neural systems process data. Artificial intelligence and cognitive modelling try to simulate some properties of biological neural networks. In the artificial intelligence field, artificial neural networks have been applied successfully to speech recognition, image analysis and adaptive control, in order to construct software agents (in computer and video games) or autonomous robots.

Historically, digital computers evolved from the von Neumann model, and operate via the execution of explicit instructions via access to memory by a number of processors. On the other hand, the origins of neural networks are based on efforts to model information processing in biological systems. Unlike the von Neumann model, neural network computing does not separate memory and processing.

Neural network theory has served to identify better how the neurons in the brain function and provide the basis for efforts to create artificial intelligence.

History [edit]

The preliminary theoretical base for contemporary neural networks was independently proposed by Alexander Bain[4] (1873) and William James[5] (1890). In their work, both thoughts and body activity resulted from interactions among neurons within the brain.

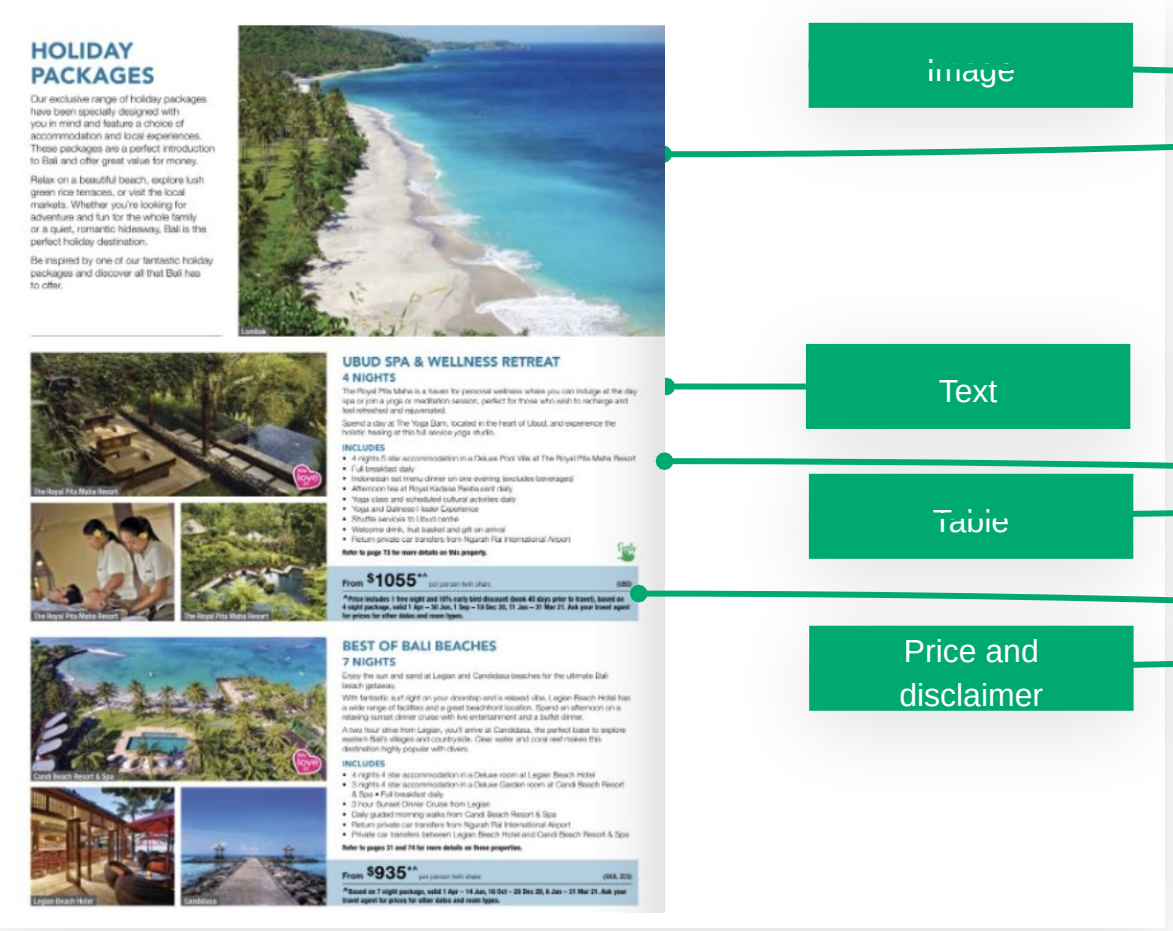
For Bain,[4] every activity led to the firing of a certain set of neurons. When activities were repeated, the connections between those neurons strengthened. According to his theory, this repetition was what led to the formation of memory. The general scientific community at the time was skeptical of Bain's[4] theory because it required what appeared to be an inordinate number of neural connections within the brain. It is now apparent that the brain is exceedingly complex and that the same brain "wiring" can handle multiple problems and inputs.

James'[5] theory was similar to Bain's,[4] however, he suggested that memories and actions resulted from electrical currents flowing among the neurons in the brain. His model, by focusing on the flow of electrical currents, did not require individual neural connections for each memory or action.

isolutions

Data Extraction and Chunking Challenges

Working with complex documents



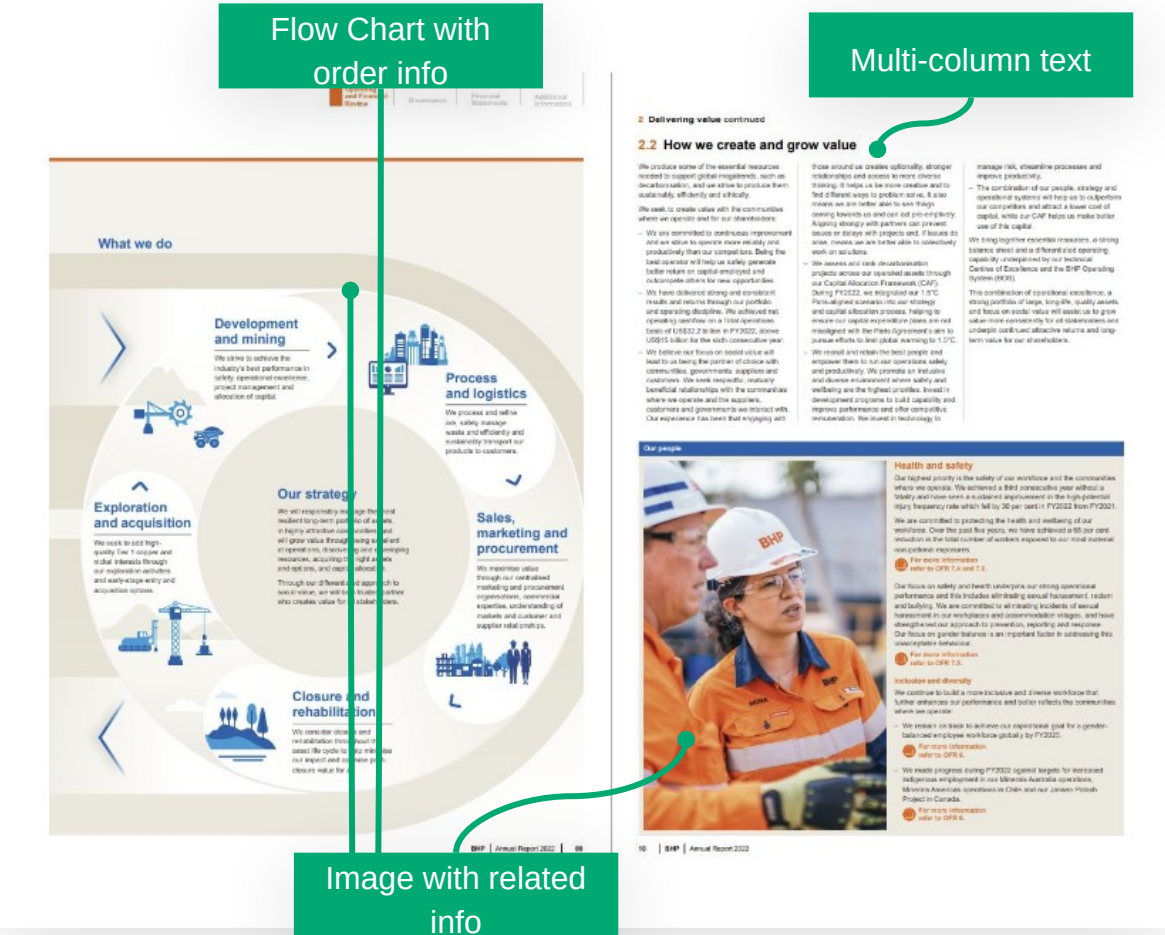
Other challenges:

- Text mixed with image
- Irregular placement of text
- Color encoded focus *(Important for context)*

isolutions

Data Extraction and Chunking Challenges

Working with complex documents



Other challenges:

- Chart with hierarchical information. Keeping the order of the information is critical.
- Multi-column text and the order of columns if crucial.
- Keeping images with related information is crucial.

isolutions

Data Extraction and Chunking Challenges

Working with different document types

- Preserving logical sections of the content might be difficult for some document types.
  - HTML requires tag based chunking to preserve logical structure of the content.
- HTML Tables are token heavy.
- Example: 3x3 small table;
  - Plain text: 11 tokens
  - JSONL: 29 tokens
  - HTML: 132 tokens (‼)

```html
<table style="border-collapse: collapse; width: 50%;">
    <tr>
        <th style="background-color: #f2f2f2; padding: 8px;">Header 1</th>
        <th style="background-color: #f2f2f2; padding: 8px;">Header 2</th>
        <th style="background-color: #f2f2f2; padding: 8px;">Header 3</th>
    </tr>
    <tr>
        <td style="border: 1px solid #ddd; padding: 8px;">Row 1, Cell 1</td>
        <td style="border: 1px solid #ddd; padding: 8px;">Row 1, Cell 2</td>
        <td style="border: 1px solid #ddd; padding: 8px;">Row 1, Cell 3</td>
    </tr>
    <tr>
        <td style="border: 1px solid #ddd; padding: 8px;">Row 2, Cell 1</td>
        <td style="border: 1px solid #ddd; padding: 8px;">Row 2, Cell 2</td>
        <td style="border: 1px solid #ddd; padding: 8px;">Row 2, Cell 3</td>
    </tr>
    <tr>
        <td style="border: 1
        <td style="border: 1
        <td style="border: 1
    </tr>
</table>
```

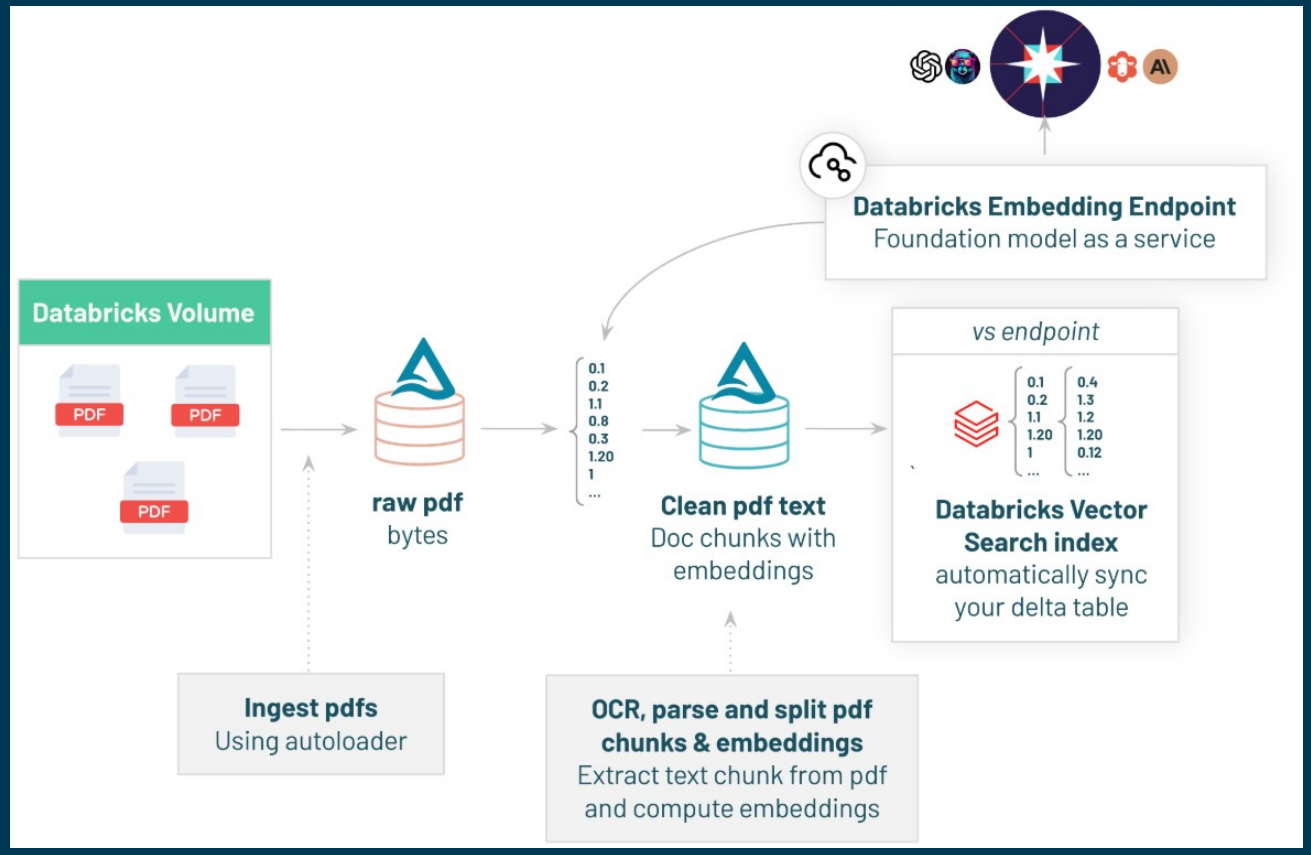
```text
Header 1    Header 2    Header 3
Row 1, Cell 1    Row 1, Cell 2    Row 1, Cell 3
Row 2, Cell 1    Row 2, Cell 2    Row 2, Cell 3
Row 3, Cell 1    Row 3, Cell 2    Row 3, Cell 3
```

```json
{
    "table": [
        {
            "Header 1": "Row 1, Cell 1",
            "Header 2": "Row 1, Cell 2",
            "Header 3": "Row 1, Cell 3"
        },
        {
            "Header 1": "Row 2, Cell 1",
            "Header 2": "Row 2, Cell 2",
            "Header 3": "Row 2, Cell 3"
        },
        {
            "Header 1": "Row 3, Cell 1",
            "Header 2": "Row 3, Cell 2",
            "Header 3": "Row 3, Cell 3"
        }
    ]
}
```

isolutions

Demo

Preparing Data for RAG



isolutions

Mosaic AI Vector Search

isolutions

What are Vector Databases?

A vector database is a database that is optimized to **store and retrieve high-dimensional vectors such as embeddings**.

In RAG architecture **contextual information** is stored in vectors.

Vector databases are designed for **efficient storage of vectors** utilized in generative AI applications, which rely on identifying documents or images with similarities.

Vector databases provide a query interface that retrieves vectors most similar to a specified query vector.

Why are Vector Databases So Hot?

Query time and scalability

- Specialized, full-fledged databases for **unstructured data**
  - Inherit database properties, i.e. Create-Read-Update-Delete (CRUD)

- Speed up query search for the closest vectors
  - Uses vector search algorithms such as Approximate Nearest Neighbor (ANN)
  - Organize embeddings into indices

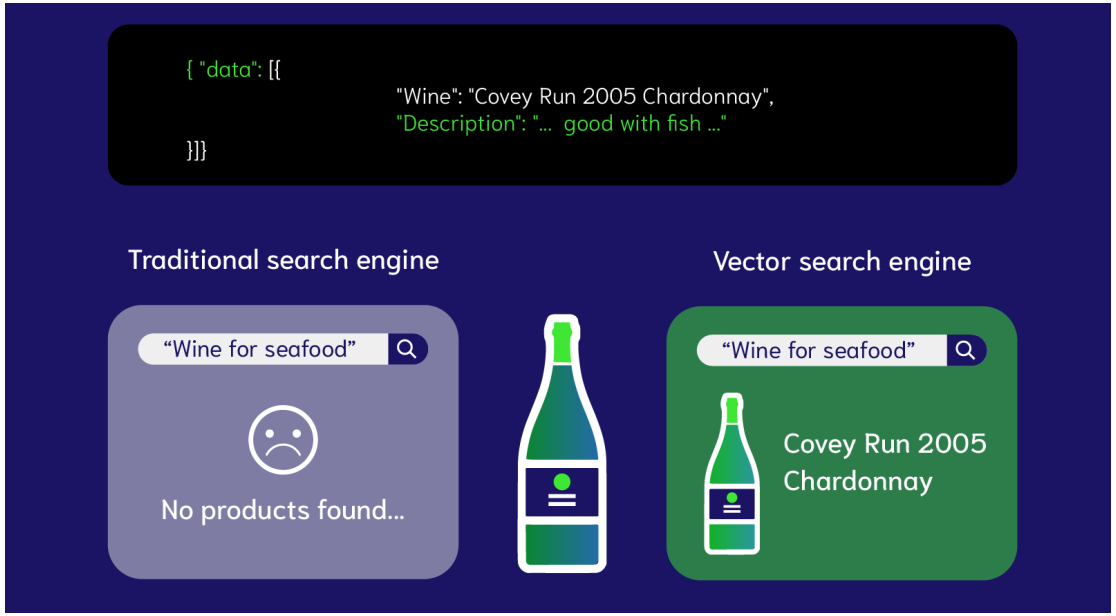
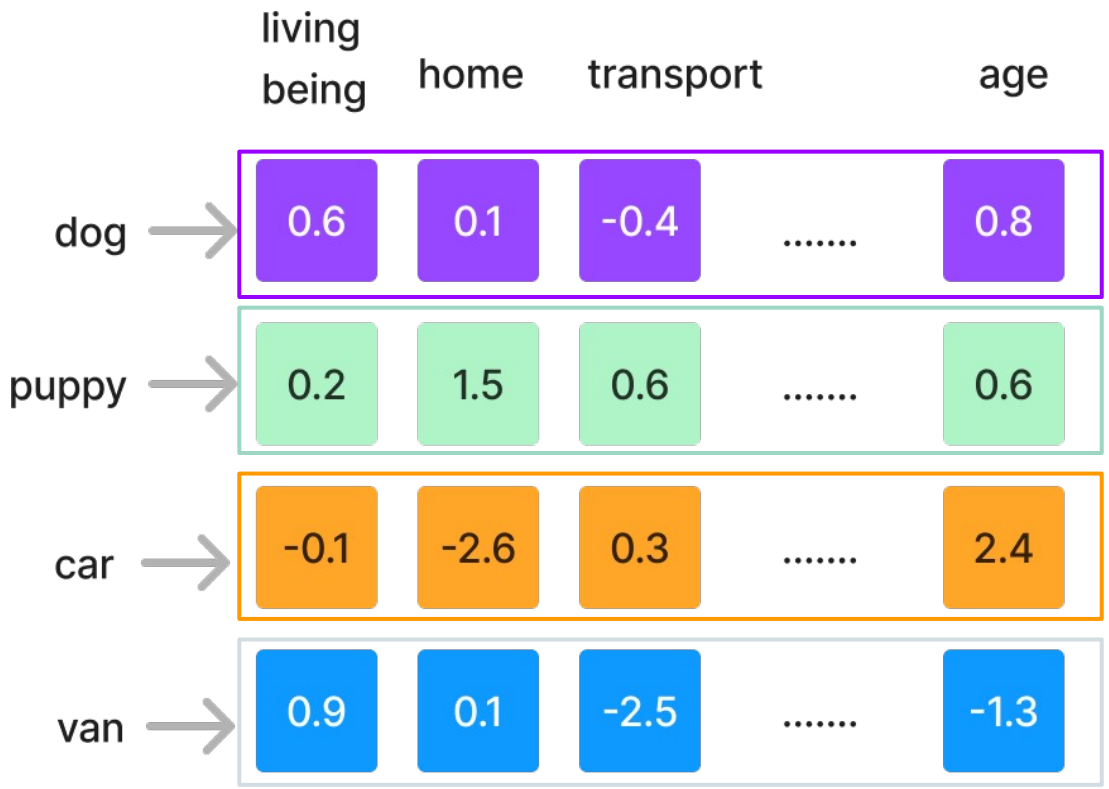
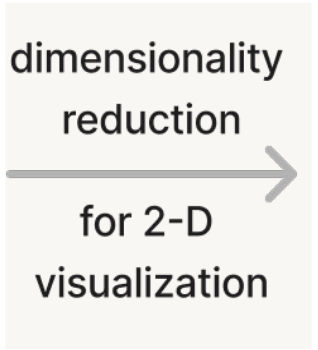


Image Source: Weaviate

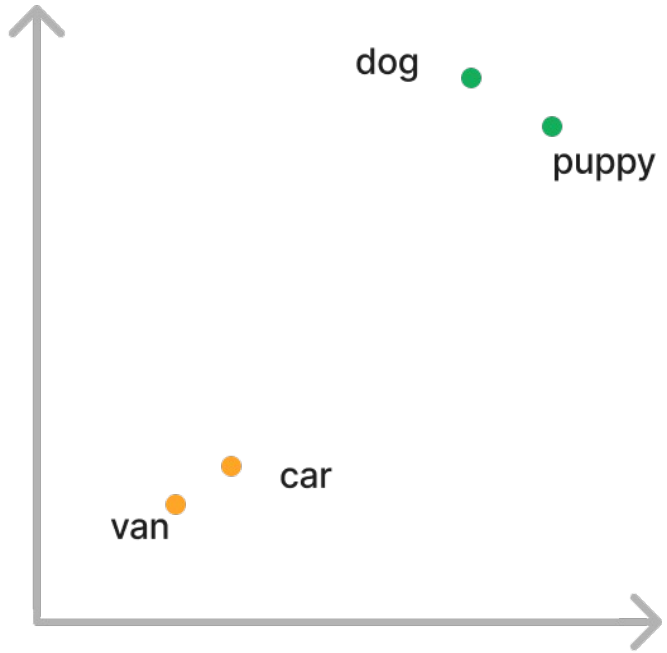
Representing Words with Vectors

Embedding: A numerical representation of content





We can project these vectors onto 2D to see how they relate graphically



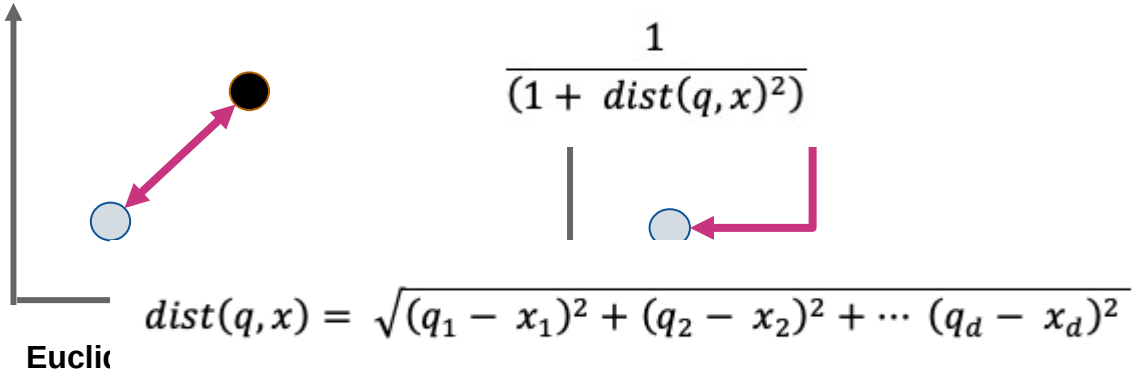
isolutions

Vector Similarity

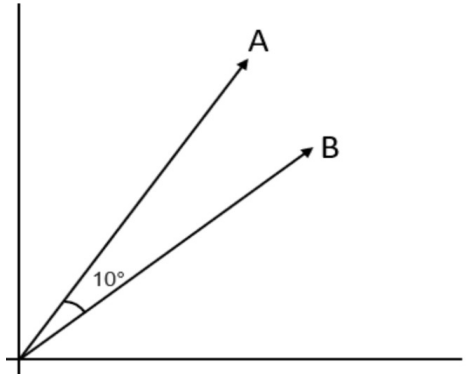
How to measure if 2 vectors are similar?

Distance Metrics

- The higher the metric, the less similar
- L2 (Euclidean) is the most popular



Similarity Metrics



The higher the metric, the *more similar*

Cosine similarity is the most popular

More info: https://docs.databricks.com/aws/en/generative-ai/vector-search

isolutions

Mosaic AI **Vector Search**

- Stores vector representation of your data, plus metadata
- Tightly integrated with your Lakehouse
- Scalable, low latency production service with zero operational overhead
- Supports ACLs using Unity Catalog integration
- API for real-time similarity search
  - Query can include filters on metadata
  - REST API and Python client

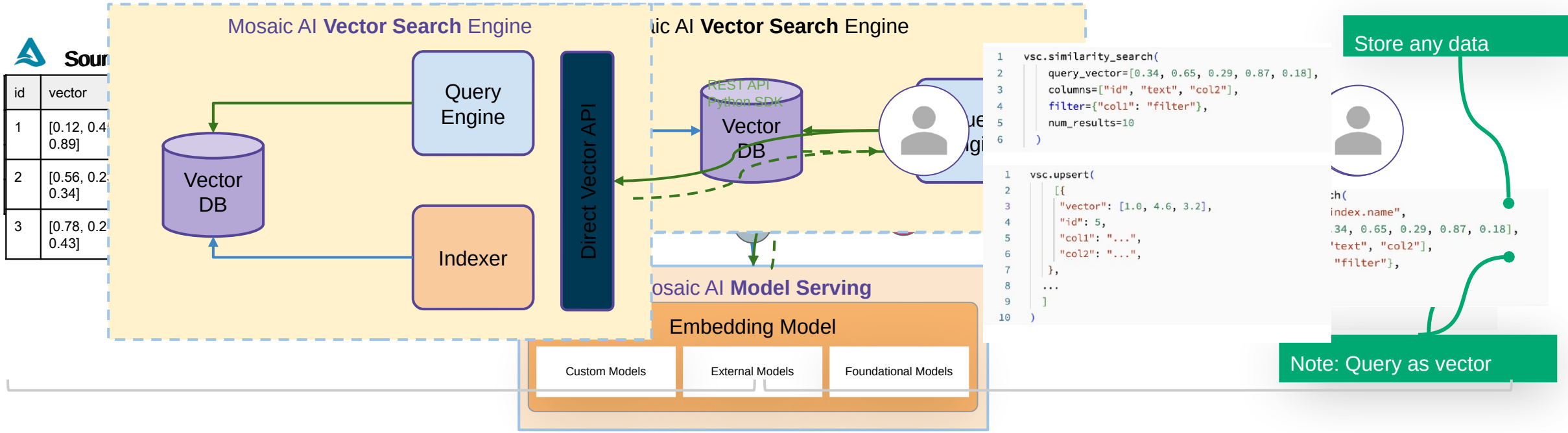
isolutions

How does Vector Search Work?

Method 1: Delta Sync API with **managed** embeddings

Method 2: Delta Sync API with **self-managed** embeddings

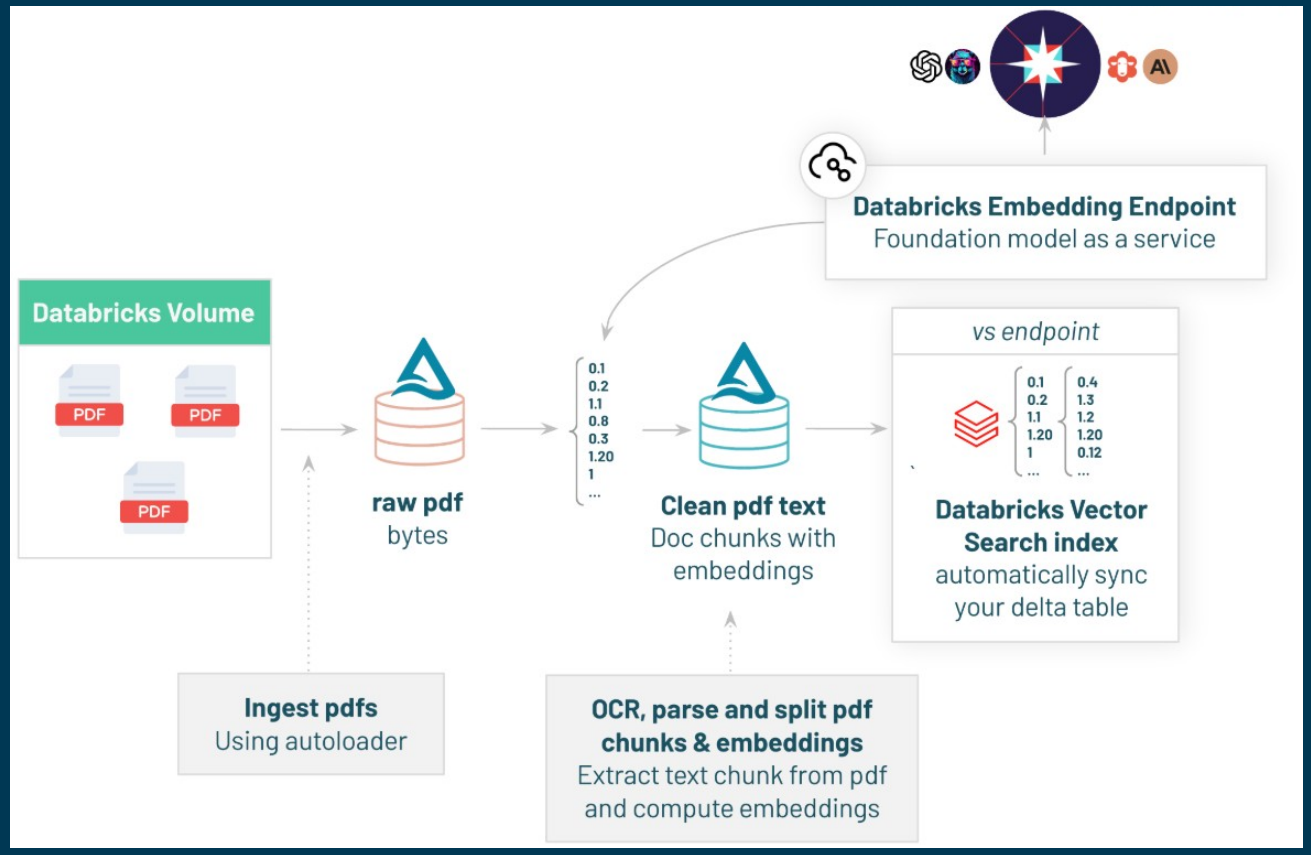
Method 3: **Direct access** CRUD API



isolutions

Demo

Create Vector Search Index



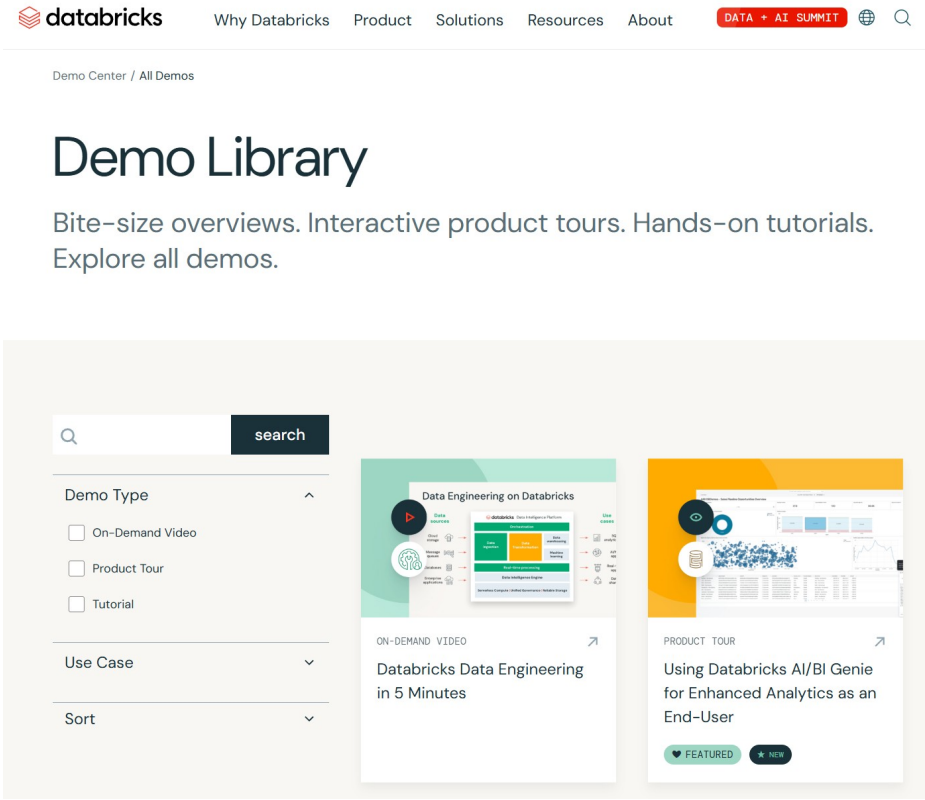
How can I reproduce it by myself?



isolutions

Find Resources Online

- Databricks Demo Library (with notebooks!):
  - Mosaic AI Demo
  - LLMs with Agents Demo
- ArXiv: https://www.alphaxiv.org/about
- Mosaic AI Vector Search requirements:
  - Unity Catalog enabled workspace.
  - Serverless compute enabled.
  - Source table must have Change Data Feed
  - To create a vector search index, you must have CREATE TABLE privileges on the catalog schema where the index will be created.



isolutions

